# DeployHub bug report

Every bug found and fixed so far, and what is still open · 26 October 2026

**Summary:** 15 bugs found, **all 15 fixed**, each now covered by an automated test. The most serious one (BUG-01) was found by you on the first real deployment: every real build on Oracle was failing. BUG-11 to BUG-15 came up while deploying your Employee Management project; your backend is now live.

## Overview

| ID | Bug | Severity | Found | Status |
|---|---|---|---|---|
| BUG-01 | Every real build failed on Oracle (ORA-01400) | High | by you, first real deployment (M6) | Fixed |
| BUG-02 | A crashing app was not recognised | Medium | by the real-Docker test suite (M6) | Fixed |
| BUG-03 | Live log streaming broke after the app was stopped and started | Medium | by the full test run (M5) | Fixed |
| BUG-04 | An older deployment could replace a newer live one | High | in the code review (before M5) | Fixed |
| BUG-05 | A replaced deployment lost its "went live" time | Low | in the code review | Fixed |
| BUG-06 | Pages stopped refreshing after one failed request | Medium | in the code review | Fixed |
| BUG-07 | The log viewer retried forever on errors that can't recover | Low | in the code review | Fixed |
| BUG-08 | The Deploy button would fail on plain http | Medium | in the code review | Fixed |
| BUG-09 | Start-command error appeared one save too late | Low | by the existing tests (M6, before release) | Fixed |
| BUG-10 | Generated Node.js builds would skip needed packages | Medium | in self-review (M6, before release) | Fixed |
| BUG-11 | Apps could not create folders in their own directory | High | by reading your repository (before you deployed it) | Fixed |
| BUG-12 | A crash during startup sometimes reported exit code 0 | Low | by the real-Docker tests | Fixed |
| BUG-13 | The address proxy could not read its own configuration | High | by the real-Docker tests (before release) | Fixed |
| BUG-14 | Apps using npm/npx at start failed with "permission denied" | High | by you, deploying your Angular frontend | Fixed |
| BUG-15 | Build failures showed a Docker Desktop link instead of the cause | Medium | by you, deploying DeployHub's own frontend | Fixed |

## Details

### BUG-01 · Every real build failed on Oracle (ORA-01400)

| | |
|---|---|
| **Severity** | **High** · Found: by you, first real deployment (M6) · Status: **Fixed** |
| **What happened** | Deploying `123sajal/Employee-Management-` failed 3 times with `ORA-01400: cannot insert NULL into DEPLOYMENT_LOGS.MESSAGE`, right after the build started. Every real build on Oracle would have failed the same way. |
| **Cause** | `docker build` prints empty lines, and DeployHub saved every output line to the log. Oracle stores an empty string as NULL, and the message column is NOT NULL. The test database (H2) stores empty strings normally, and the Docker tests used a fake log writer, so no test hit it. |
| **Fix** | Blank lines are skipped (they carry no information) in both the log writer and the build-output handler. |
| **Guarded by** | `DeploymentLifecycleIntegrationTest.blankLogLinesAreSkippedInsteadOfBreakingTheDeployment`, which also runs on real Oracle. |

### BUG-02 · A crashing app was not recognised

| | |
|---|---|
| **Severity** | **Medium** · Found: by the real-Docker test suite (M6) · Status: **Fixed** |
| **What happened** | An app that exits at once (e.g. a missing file) was reported as "did not answer within 60 s" after waiting the full minute, instead of failing at once with its exit code. |
| **Cause** | Containers run with `--restart unless-stopped`, so Docker restarts a crashed app immediately and "is it running?" kept answering yes. |
| **Fix** | A crash is detected from Docker's restart count and status; the app's last output is copied into the log. |
| **Guarded by** | `DockerExecutorDockerTest.anAppThatCrashesFailsTheDeploymentAndLeavesNothingRunning` (real Docker). |

### BUG-03 · Live log streaming broke after the app was stopped and started

| | |
|---|---|
| **Severity** | **Medium** · Found: by the full test run (M5) · Status: **Fixed** |
| **What happened** | After a restart of the application context, every new live-log stream failed to open. |
| **Cause** | The stream service shut its thread pool down on stop() but only created it once, in its constructor. |
| **Fix** | The thread pool is created in start() and shut down in stop(). |
| **Guarded by** | `LogStreamIntegrationTest.streamingWorksAgainAfterTheApplicationIsStoppedAndStarted`. |

### BUG-04 · An older deployment could replace a newer live one

| | |
|---|---|
| **Severity** | **High** · Found: in the code review (before M5) · Status: **Fixed** |
| **What happened** | If an older deployment finished late (e.g. after retries), it went LIVE and replaced newer code. Two deployments finishing at the same moment could also both stay LIVE. |
| **Cause** | Going LIVE superseded "any other LIVE deployment" without checking which one was newer, and without a lock. |
| **Fix** | Going LIVE locks the project row; an older deployment is skipped (CANCELLED: "newer deployment #N is already live"). |
| **Guarded by** | `DeploymentLifecycleIntegrationTest.olderDeploymentFinishingLateNeverReplacesNewerLiveOne` (H2 and Oracle). |

### BUG-05 · A replaced deployment lost its "went live" time

| | |
|---|---|
| **Severity** | **Low** · Found: in the code review · Status: **Fixed** |
| **What happened** | When a deployment was superseded, its "Finished" time changed to the moment it was replaced. |
| **Cause** | The finish time was set again on every final status, including SUPERSEDED. |
| **Fix** | The finish time is set only once. |
| **Guarded by** | `newLiveDeploymentSupersedesPreviousOne` checks it is unchanged. |

### BUG-06 · Pages stopped refreshing after one failed request

| | |
|---|---|
| **Severity** | **Medium** · Found: in the code review · Status: **Fixed** |
| **What happened** | After a short backend outage, the dashboard and deployment pages froze until reloaded; the project page silently showed "no deployments". |
| **Cause** | An error ended the refresh loop for good. |
| **Fix** | Temporary errors (API down, 5xx, 429) are shown and refreshing continues; errors like 404 stop it. |
| **Guarded by** | Frontend tests for the dashboard and the polling helper. |

### BUG-07 · The log viewer retried forever on errors that can't recover

| | |
|---|---|
| **Severity** | **Low** · Found: in the code review · Status: **Fixed** |
| **What happened** | For a deleted deployment (404) the log panel kept retrying every 3 seconds forever. |
| **Cause** | Every error was treated as a lost connection. |
| **Fix** | It stops and shows the error for 404 and similar; it only reconnects for temporary problems. |
| **Guarded by** | Log viewer test "stops for good on an error that reconnecting cannot fix". |

### BUG-08 · The Deploy button would fail on plain http

| | |
|---|---|
| **Severity** | **Medium** · Found: in the code review · Status: **Fixed** |
| **What happened** | On a page served over http from a server address (not localhost), pressing Deploy would throw an error. |
| **Cause** | `crypto.randomUUID()` only exists on secure pages (https or localhost). |
| **Fix** | Falls back to `crypto.getRandomValues()`, which works everywhere. |
| **Guarded by** | Idempotency-key tests (secure page and fallback). |

### BUG-09 · Start-command error appeared one save too late

| | |
|---|---|
| **Severity** | **Low** · Found: by the existing tests (M6, before release) · Status: **Fixed** |
| **What happened** | When a project form had several mistakes, a missing start command was only reported after the others were fixed. |
| **Cause** | The new "optional for Dockerfile projects" rule was checked after the other validations, not with them. |
| **Fix** | Made it a class-level validation rule, so all errors are reported together. |
| **Guarded by** | `ProjectApiIntegrationTest.rejectsInvalidFieldsWithProblemDetail`. |

### BUG-10 · Generated Node.js builds would skip needed packages

| | |
|---|---|
| **Severity** | **Medium** · Found: in self-review (M6, before release) · Status: **Fixed** |
| **What happened** | Builds needing development packages (TypeScript, Vite, test tools) would have failed in `npm ci`. |
| **Cause** | The generated Dockerfile set `NODE_ENV=production` for the build, which makes npm skip dev dependencies. |
| **Fix** | Removed it from the generated Dockerfile. |
| **Guarded by** | `DockerfileGeneratorTest` checks it is absent. |

### BUG-11 · Apps could not create folders in their own directory

| | |
|---|---|
| **Severity** | **High** · Found: by reading your repository (before you deployed it) · Status: **Fixed** |
| **What happened** | Your backend creates an `uploads/` folder at startup. In a DeployHub container that would have failed with "permission denied" and the app would have crashed. |
| **Cause** | Apps run as an unprivileged user for safety, but their files were copied in as owned by root. |
| **Fix** | The generated Dockerfile gives the app folder to the app's user (`COPY --chown`). Files an app writes are still lost on redeploy, as on other platforms. |
| **Guarded by** | `DockerExecutorDockerTest`: the test app creates `uploads/` at startup (real Docker). |

### BUG-12 · A crash during startup sometimes reported exit code 0

| | |
|---|---|
| **Severity** | **Low** · Found: by the real-Docker tests · Status: **Fixed** |
| **What happened** | When an app crashed while starting, the message sometimes said "exit code 0" instead of the real code. |
| **Cause** | With automatic restarts on, Docker had already restarted the app, resetting its exit code, when DeployHub looked. |
| **Fix** | Automatic restarts are switched on only after the health check passes. |
| **Guarded by** | `anAppThatCrashesFailsTheDeploymentAndLeavesNothingRunning` (real Docker, repeated 3 times). |

### BUG-13 · The address proxy could not read its own configuration

| | |
|---|---|
| **Severity** | **High** · Found: by the real-Docker tests (before release) · Status: **Fixed** |
| **What happened** | The fixed project address did not answer: the proxy container crashed and restarted in a loop. |
| **Cause** | Java creates temporary files readable by their owner only; `docker cp` kept that, and the hardened proxy runs without the capability to bypass file permissions. |
| **Fix** | The configuration file (container names only, no secrets) is made readable before it is copied in. |
| **Guarded by** | `aProjectAddressServesTheAppAndRoutesApiToAnotherAppWithItsEnvironment` (real Docker). |

### BUG-14 · Apps using npm/npx at start failed with "permission denied"

| | |
|---|---|
| **Severity** | **High** · Found: by you, deploying your Angular frontend · Status: **Fixed** |
| **What happened** | The frontend's start command `npx serve ...` failed at once with npm error EACCES on `/tmp/.npm`. |
| **Cause** | The build ran as root with the same home folder the app uses later (`/tmp`), so `npm ci` left a root-owned cache there that the app's unprivileged user couldn't write to. |
| **Fix** | The generated Dockerfile sets the app's home folder only after the build step, so the build keeps its files in root's own home. |
| **Guarded by** | `whatTheBuildLeavesInItsHomeFolderDoesNotBlockTheAppsHomeFolder` (real Docker). |

### BUG-15 · Build failures showed a Docker Desktop link instead of the cause

| | |
|---|---|
| **Severity** | **Medium** · Found: by you, deploying DeployHub's own frontend · Status: **Fixed** |
| **What happened** | The failure message was "View build details: docker-desktop://...", which says nothing about what went wrong (the real cause was `npm error Missing: @emnapi/runtime ... from lock file`). |
| **Cause** | The message used the last output line, and Docker prints its own summary and that link after the real error. |
| **Fix** | The message now shows the first real error line of the build, skipping Docker's summary lines; more output lines are kept so it is still there. |
| **Guarded by** | `aBuildFailureReportsTheRealErrorNotDockersSummaryOrDesktopLink`. |

## Still open: known limitations (not bugs, by design for now)

| Area | Limitation | Planned |
|---|---|---|
| Live apps | A LIVE app isn't watched after its health check. Docker restarts it if it crashes, but DeployHub still shows LIVE. | Later |
| Addresses | Project addresses (http://localhost:19xxx) work on this Mac only; no HTTPS or public hostname yet. | M7 |
| App files | Files an app writes (e.g. uploads) are lost when it is redeployed. | Use a database |
| Builds | Builds have a 15-minute limit but no memory limit of their own. | Later |
| Resources | Docker has ~4 GB, shared with Oracle (~2 GB). Several apps live at once can run out of memory. | Your choice |
| Rollback | Deployments made before real builds recorded no exact commit; rolling back to them deploys the branch's current code. | None (old data) |
| Queue | The per-user limit of 5 is checked without a lock; a burst can briefly exceed it. | Later |
| Sessions | Logins live in the backend's memory: restarting it logs everyone out. | M7 |
| Repositories | Public repositories only; no build on push. | Only with your go-ahead |
| Secrets | Environment variables can't be used during the build, and the encryption key can't be rotated yet. | Later |

## Not yet verified

| What | Why | Next step |
|---|---|---|
| Your Employee Management project end to end (MySQL, backend, frontend with /api route) | All parts are tested with small sample apps in real Docker, but not with your project. I was not permitted to build external code myself. | Follow the steps in the README |
| New screens in a browser (environment variables, routes, address, metrics, rollback) | Covered by automated tests, but not looked at by eye. | Check while testing |

Harmless messages you may see: Maven prints "Could not transfer metadata ... HTTP Status: 401" for two optional Flyway download sites; and Docker Desktop may label the Oracle container "Paused" while its Resource Saver is active, even though it is running.